

Don't Trim the Tail: Sequence Truncation Weakens Threshold Learning for Rare Control Tokens

Choe Chol Hun (Independent Researcher, cch992@gmail.com)

Abstract

Fine-tuned language models are increasingly trained to emit rare, machine-consumed control markers — tokens that end a session, invoke a tool, or trigger a refusal — on a semantic threshold condition. We show that the shape of the supervised fine-tuning data, with model family and training budget held fixed, has a first-order effect on both when such a marker fires and what reason the model encodes for firing. In a fully crossed factorial study of marker position, marker format, and sequence trimming, we find that trimming each training sequence to end at the marker — a practice one might expect to sharpen marker learning — instead raises premature firing by 0.44 to 0.58 across every design, the largest main effect in the study. The evidence is most consistent with a threshold-laxity mechanism: trimming deletes the training turns that show the trigger present but not yet followed by firing, so the model stops thresholding on the trigger's magnitude, and retaining a benign post-marker continuation substantially reduces premature firing. Separately, an axis-typed marker acts as a semantic gate, attributing the correct violated invariant on the large majority of fires, including under a distractor, whereas a generic marker carries no such signal. The effect replicates in direction in a second domain and model family. Together these yield cheap data-curation principles for reliable control-token emission.

Keywords: large language models; supervised fine-tuning; data curation; control tokens; sequence truncation; threshold learning; tool calling; agentic systems.

1. Introduction

Large language models increasingly serve as decision-making components inside larger software systems rather than as standalone text generators, coordinating external tools, agents, retrieval pipelines, and dialogue workflows through **rare structured control markers** — tokens that signal an external system to take an action: end a session, hand off to a tool, refuse, or escalate. Unlike ordinary generation, a control marker is an *execution signal*: it has a *trigger condition* (fire when, and only when, some semantic threshold is met) and its value is consumed by machinery, not read by a human. Emitting one too early, too late, or under the wrong conditions can alter system behavior even when the surrounding text remains fluent — so getting the *timing* wrong and the *content* wrong (firing without a usable reason) are distinct failure modes with distinct costs.

This paper shows that both failure modes are strongly influenced by the **shape of the SFT data** — with model family and training budget held fixed — and that two common curation choices have large, measurable, and in one case counter-intuitive effects.

Contribution 1: the terminal-position / trim artifact. A common practice when training a model to emit a rare marker is to *trim* each training sequence to end at the marker; one might expect this to sharpen marker learning by removing distracting continuation. We show it does the opposite. Trimming deletes every training example in which the marker's trigger feature is present but *not* followed

by firing, leaving the trigger perfectly predictive of the marker and impairing the model’s ability to threshold on the trigger’s magnitude. In a fully crossed $2 \times 2 \times 2$ (position $\times$ marker-format $\times$ trim) design on a four-axis session-ending task, trimming raises premature-firing rate by **+0.44 to +0.58 across every design tested** — the largest observed main effect in the tested design, confirmed by an item-level factorial logistic model in which trim is the largest term by a wide margin (odds ratio $\approx \$9.6$) and every interaction with position and marker format is non-significant. Retaining the benign post-marker continuation substantially reduces premature firing.

Contribution 2: typed markers as semantic gates. A *typed* marker ([SESSION_END: <axis>]) forces the model to attribute firing to a specific violated invariant; a *generic* marker ([SESSION_END]) does not. Typed-trained models name the correct axis on **94–99%** of fires and never emit a contentless marker, turning session-ending into an accurate, actionable classification; generic-trained models carry no such signal by construction. Attribution is robust to the trim manipulation even where timing is substantially worsened, establishing that *when* a model fires and *why* it says it fires are separable, separately curated behaviors. Typed markers support robust per-axis attribution under a sub-threshold distractor: with a second, sub-threshold violation present in the conversation, typed models still name the axis at threshold on 96–98% of fires and are pulled to the distractor under 2% of the time.

Contribution 3 (generalization): the effect is not specific to the tutoring corpus. We replicate trim→premature *in direction* in a **different domain on a different model family** — fine-tuning Llama-3.2-1B-Base on a synthetic customer-support escalation task, where the agent must emit an escalation marker after the third *explicit* escalation request (angry-but-non-requesting messages are distractors that must not count). The trimmed model fires the escalation marker prematurely more than the untrimmed one (0.830 vs 0.683, both at full recall), the same direction as the tutor result. This suggests the vulnerability is a property of next-token training on trimmed sequences at a rare, count-triggered semantic marker, not an artifact of one dataset or one model lineage; we make the narrower directional claim rather than a claim of universal magnitude, since both settings are small synthetic tasks (§6.6).

Together these results form **data-curation principles for rare control tokens**. We additionally sketch a candidate remedy — annotating the marker with its strike count to make the latent counter an explicit supervised target — and leave its evaluation to future work (§6.6).

2. Related Work

Control tokens and structured generation. Fine-tuned LMs are routinely trained to emit special tokens that gate downstream machinery — end-of-turn and stop tokens, tool-call and function-call delimiters, refusal/safety triggers, and routing tags. Recent function-calling work focuses on *scaling and verifying* the SFT data that teaches these tokens (Schick et al. 2023; Qin et al. 2024; Liu et al. 2024; Liu et al. 2025), and deployed agent benchmarks show how much reliable control-token emission matters in multi-turn tool use (Yao et al. 2024). A parallel line enforces structure at *decode* time via grammar-constrained or guided generation (Willard and Louf 2023; Dong et al. 2024; Park et al. 2024) — though constraining the decoder distorts the learned distribution, motivating our focus on shaping the *training data* instead. Most of this work treats the markers as generation *targets* and studies *whether* the model emits them; we instead study how the *shape* of the training sequences around a rare marker determines *when* (timing/threshold) and *with what content* (attribution) the model emits it.

Sequence termination, EOS, and stop-token learning. A rare control marker that ends a session is closely related to the end-of-sequence token: both are learned signals that terminate generation on a learned condition. Work on end-of-sequence and stop-token behavior notes that models can acquire biased termination tendencies from the length and position statistics of training sequences — over- or under-terminating relative to the intended condition (Newman et al. 2020; Stern et al. 2019). Our trim manipulation is a controlled instance of this: truncating every marker-bearing training sequence *at* the marker maximizes the correlation between the marker and sequence-end, and we measure the resulting shift in the emission threshold directly.

Truncation, loss masking, and preprocessing artifacts in SFT. How each training example is pre-processed — where it is truncated to fit the context window, whether loss is masked to response tokens only, and how examples are packed — is known to affect fine-tuning outcomes, yet is frequently left as an undocumented pipeline detail (Raffel et al. 2020; Muennighoff et al. 2023). Response-only loss masking and sequence packing in particular change which tokens supply gradient and what context each target is conditioned on (Wolf et al. 2020). We treat one such choice — post-marker truncation — as a first-class experimental factor rather than an incidental preprocessing step, and mask loss to assistant turns throughout so masking is held constant across cells.

Premature and false-positive control actions. In deployed agentic systems the operational failure our probe measures — firing a control marker *before* its trigger condition — appears as premature or spurious tool invocation, over-eager function calls, and mis-calibrated refusal/safety triggers, all of which degrade reliability even when surrounding text is fluent (Yao et al. 2024). Rare-event and selective-prediction work frames the same tension as calibrating *when to abstain or act* on a low-base-rate trigger (Geifman and El-Yaniv 2017; El-Yaniv and Wiener 2010). We connect this deployment-level failure to a specific, controllable data-curation cause.

Shortcut learning and spurious correlations. Models minimize loss via the cheapest sufficient predictor, latching onto features that are predictive in the training distribution but not causal for the task. Our trim result is a control-token instance: trimming makes *sequence-terminality / escalation- presence* perfectly predictive of the marker, and the model binds to that cheap cue instead of the true count-based trigger (Geirhos et al. 2020; McCoy et al. 2019; Gururangan et al. 2018).

Data curation for fine-tuning. A large body of work studies which *examples* to include (quality filtering, dedup, mixture weights). Less attention is paid to how each example is *shaped* — where it is truncated, what is masked, whether post-target continuation is retained. We isolate one such choice (post-marker trimming) and show it has a first-order effect on behavior, larger than the architectural/position design choices it is usually bundled with (Zhou et al. 2023; Muennighoff et al. 2023).

Counting and multi-turn state in LMs. Emitting a marker “on the third strike” requires maintaining a count across turns. Prior work shows LMs struggle with exact counting and that making intermediate state explicit (scratchpads, chain-of-thought) helps. The candidate count-annotated marker we outline as future work (§6.6) is a minimal, inference-cheap form of this — supervising the running count directly in the output rather than in a separate reasoning trace — in the spirit of process-supervision work that supervises intermediate steps rather than only the final answer (Bhattamishra et al. 2020; Nye et al. 2021; Wei et al. 2022; Lightman et al. 2024; Zheng et al. 2024).

The gap we address. These threads — instruction tuning, alignment, control-token utilization, termination bias, and preprocessing artifacts — are individually well studied, but to our knowledge they have not been brought together to isolate the effect of *how each training sequence is shaped around a rare marker* — specifically whether truncating at the marker changes the learned emission threshold — while holding token design, position, and optimization fixed. As a result it is, as far as we are

aware, not yet established whether premature control-token emission originates from token design, optimization, or sequence structure. (We make this a bounded claim rather than an assertion of absolute novelty, pending a more exhaustive survey.) We address this by making post-marker trim an explicit factor in a fully-crossed $2 \times 2 \times 2$ design (§3), so its effect is measured independently of the position and marker-format choices it is usually bundled with.

Scope of this study. We take a multi-turn persistence marker — a session-end sentinel that must fire on the third same-axis violation — as a controlled testbed, because it is a rare, count-triggered, machine-consumed control token of exactly the kind whose curation we study. Prior work establishes that such multi-turn behaviors are acquired through supervised fine-tuning rather than prompting alone; we take that acquisition as given and ask a distinct, downstream question: once training is committed to, which *data-shape* choices — where a sequence is trimmed, whether the marker is axis-typed, whether the count is supervised — govern *when* the marker fires and *what reason* it encodes. Our claims are therefore about the training signal for control-token emission, and are independent of the tutoring domain we draw the testbed from.

3. Method

3.1 Task: a rare structured control marker

The study task is a four-axis **session-ending sentinel**. A tutor model converses in character; on four governed abuse axes — off-topic safety probes, sustained L1/refusal-to-use-English, sustained persona attacks, and sustained role-swap — the third *same-axis* violation must trigger a literal marker that an external dispatcher consumes to close the session:

```
persistent_off_topic          -> [SESSION_END: persistent_off_topic]
persistent_language_violation -> [SESSION_END: persistent_language_violation]
persistent_persona_break      -> [SESSION_END: persistent_persona_break]
persistent_role_swap          -> [SESSION_END: persistent_role_swap]
```

The marker is *rare* (one firing per governed dialogue, absent from $\sim 4/5$ of the corpus), has a *count-based trigger* (exactly the third same-axis strike), and is *machine-consumed* (its axis label routes downstream handling). These three properties make it a clean testbed for control-token curation.

3.2 Factors under study

We manipulate two data-shape factors, fully crossed, at matched training budget (1 epoch, QLoRA (Dettmers et al. 2023) SFT, 0.8B base):

- **Position design** — the trained firing turn is either **fixed** (always turn 7) or **4-variant decorrelated** (turns {5,7,9,11}).
- **Marker format** — **typed** ([SESSION_END: <axis>], carries the axis) or **generic** (bare [SESSION_END], no axis).

This yields four cells: **A1** (4var+typed), **A5** (fix7+typed), **A6** (fix7+generic), **A7** (4var+generic).

Crossed orthogonally with a third factor:

- **Post-marker trim** — each persistent training record is either **trimmed** to end at the marker, or **untrimmed** (the benign post-marker continuation is retained). All four cells exist in both variants, giving a **fully crossed $2 \times 2 \times 2$ factorial design yielding eight conditions**:

#	Position	Marker format	Trim status	Cell
1	4-variant	typed	untrimmed	A1 (untrim)
2	4-variant	typed	trimmed	A1 (trim)
3	fixed-7	typed	untrimmed	A5 (untrim)
4	fixed-7	typed	trimmed	A5 (trim)
5	fixed-7	generic	untrimmed	A6 (untrim)
6	fixed-7	generic	trimmed	A6 (trim)
7	4-variant	generic	untrimmed	A7 (untrim)
8	4-variant	generic	trimmed	A7 (trim)

Every other factor is held constant across all cells and variants — base model, tokenizer, optimizer, learning-rate schedule, batch size, epoch budget, seed policy, and the shared evaluation prompts — so any behavioral difference is attributable to the three data-shape factors above, and trim in particular is isolated by construction (only the post-marker continuation differs; §4.4 verifies the trimmed and untrimmed records share an exact prefix).

3.3 The trim manipulation (the primary lever)

Untrimmed record



Trimmed record



Both records share an identical prefix; only the post-marker tail differs.

Figure 1: Untrimmed vs. trimmed training record. Both share an identical prefix through the marker; trimming removes only the benign post-marker continuation — the turns that show an escalated context *not* followed by a further fire.

A trimmed record ends at the marker; every escalated conversation in trimmed training therefore *terminates* at a fire. An untrimmed record keeps the turns after the marker (the tutor continues benignly), supplying examples in which an escalated context is present but *not* followed by a fire. Our hypothesis (§3.6) is that trimming deletes exactly the counterexamples the model needs to learn that escalation is necessary but not *sufficient* — only the third strike is.

3.4 Metrics

- **Recall** — on true third-strike positives, did the marker fire?
- **Premature-firing rate** — on sub-threshold contexts (1–2 strikes), did it fire anyway? This is the failure mode trim is hypothesized to induce. Stratified by violation count (vc=1 vs vc=2) and by turn.

- **Axis-attribution accuracy** — of typed fires, the fraction naming the *correct* axis. Undefined for generic markers by construction; that asymmetry is itself the semantic-gate result.

3.5 Probes

- **persistent_probe** — true third-strike positives (recall).
- **persistent_premature_probe** — single-axis sub-threshold contexts (premature firing), $n=318$, stratified by (vc, turn).
- **mixed_violation_probe** (this paper) — the mixed-axis distractor attribution test: primary axis X escalates to threshold while a distractor axis Y appears sub-threshold. `fire_correct` records (X at 3rd strike) test correct attribution under distraction; `distractor_sub` records (X at 2nd strike, Y present) test that mere escalation of *some* axis does not trigger firing. $n \approx 282(141 \text{ fire}_{correct} / 141 \text{ distractor}_{sub})$, all 12 $X \times Y$ axis pairs covered.

3.6 Hypotheses

- **H1 (trim $\rightarrow$ premature).** Trimming raises premature-firing rate in every (position $\times$ marker) cell, by an amount larger than either design factor.
- **H2 (typed = semantic gate).** Typed models attribute the correct axis at high accuracy and never emit a contentless marker; generic models cannot attribute at all. Attribution is robust to trim even where timing is not.

We additionally sketch a candidate remedy — supervising the strike count directly in the marker — as future work rather than a tested hypothesis (§6.6).

4. Experimental Setup

4.1 Base model, training regime

All conditions fine-tune the same **Qwen3.5 0.8B base** with QLoRA (Dettmers et al. 2023; Hu et al. 2022) SFT (no DPO (Rafailov et al. 2023)), **1 epoch, seed 42**, via `scripts/run_training.py --stages train_sft` reading a per-condition YAML under `config/paper_v2/`. Using SFT-only at a single matched budget removes training-budget and DPO as confounds, isolating the data-shape factors. (The paper_v2 primary A1 was 2 epochs; we do not use it here — we use the matched 1-epoch retrain `training_a1_1ep.yaml`, so every cell in this paper shares budget and seed.)

4.2 The eight cells (2 $\times$ 2 $\times$ 2)

The design crosses **position** (fixed-7 / 4-variant), **marker** (typed / generic), and **trim** (trimmed / untrimmed). Cell tags follow paper_v2:

Cell	position	marker	config
A1	4-variant	typed	<code>training_a1_1ep.yaml</code>
A5	fixed-7	typed	<code>training_a5_fixed_turn_7.yaml</code>
A6	fixed-7	generic	<code>training_a6_generic_sentinel.yaml</code>
A7	4-variant	generic	<code>training_a7_generic_sentinel.yaml</code>

The trim factor is a data manipulation applied to each cell’s persistent streams (§4.4).

4.3 Data

SFT data lives in `data/`. Each condition trains from a `data/sft_filtered_*` directory holding all twelve streams (normal + generic redirect + 6 specialized redirects + 4 persistent). Only the **four persistent streams** (`persistent_off_topic`, `persistent_language_violation`, `persistent_persona_break`, `persistent_role_swap`) differ across cells; the other eight streams are shared, so any cell-to-cell difference is attributable to the persistent-stream manipulation alone. Persistent records are $\sim 1/5$ of the corpus; the remaining $\sim 4/5$ are deep benign dialogues that end normally without a marker — so “conversation depth” alone is already decorrelated from firing by the shared majority data (relevant to the mechanism, §6).

Exact corpus composition. The full stream-level composition of the untrimmed base corpus (`data/sft_filtered`), counted by `scripts/score_corpus_composition.py`:

Stream	Dialogues	Assistant turns	~Tokens	Marker-positive	Mean asst turns
normal	1399	9464	414990	0	6.76
generic redirect	496	3337	121897	0	6.73
specialized redirect — topic	132	853	32802	0	6.46
specialized redirect — language	119	774	28305	0	6.50
specialized redirect — persona	141	976	38866	0	6.92
specialized redirect — role-swap	149	1031	36885	0	6.92
specialized redirect — locale	150	1028	36142	0	6.85
specialized redirect — pedagogy	142	1010	39564	0	7.11
persistent off-topic	130	642	15639	130	4.94
persistent language	162	825	20069	162	5.09
persistent persona	176	937	25194	176	5.32
persistent role-swap	146	741	18587	146	5.08
TOTAL	3342	21618	828940	614	—

Tokens are a reproducible whitespace-token proxy over assistant turns (a model-tokenizer count would differ in absolute value but not in the composition it implies). Only the 614 persistent, marker-positive dialogues carry the sentinel; the trim manipulation touches only these.

Removed-tail statistics (the trim delta). Over all 614 matched persistent dialogues, the trim removes exactly the post-marker continuation and nothing else: mean/median **1** post-marker assistant turn removed per dialogue (min 1, max 1), mean **14.3** ~tokens (median 13, min 3, max 39). The pre-marker context and the marker turn itself are byte-identical between the trimmed and untrimmed corpora. Equivalently, tabulating assistant turns in the persistent streams by their firing target:

Corpus	Pre-marker (no fire)	Marker turn (fire)	Post-marker (no fire)
untrimmed	1917	614	614
trimmed	1917	614	0

The trim’s *entire* effect on the training distribution is to delete the 614 post-marker “escalated-context-but-no-further-fire” assistant turns — exactly the sub-threshold counterexamples the mechanism of §6.1 turns on. This makes precise, and bounds, a natural concern that trimming changes many things at once: with pre-marker context and marker turn held identical, the only manipulated quantity is the presence of these counterexamples (and the ~ 14 tokens they carry).

4.4 How each cell’s persistent data is produced (exact provenance)

The trim/marker/position variants are produced by mechanical transforms of a common source, not independent regenerations — so a cell-to-cell contrast is a clean single-factor manipulation. The chains, from the actual build scripts:

Marker (typed → generic): - `scripts/convert_a1_to_a7.py` — takes A1’s 4-variant **typed** persistent data (`data/sft_filtered`), string-replaces `[SESSION_END: persistent_<axis>]` → `[SESSION_END]`, and re-renders the deployment system prompt with the generic `[persistence]` block (`QWEN_TUTOR_SENTINEL_FORMAT=generic`). Produces A7’s persistent data. Holds position (4-variant) fixed. - `scripts/convert_a5_to_a6.py` — same transform on A5’s fixed-7 typed data → A6. Holds position (fixed-7) fixed. So $A5 \leftrightarrow A6$ and $A1 \leftrightarrow A7$ each isolate exactly the marker factor.

Trim (untrimmed → trimmed): - `scripts/trim_persistent_post_sentinel.py` — truncates each persistent record so its last message is the assistant turn containing the marker, and updates `metadata.generation.message_count`. Applied to A5/A6/A7 persistent dirs. This is the *native* trim state of A5/A6/A7. - A1’s native persistent data (`data/sft_filtered`) is **untrimmed** (keeps the benign post-marker continuation). Its trimmed counterpart is a separate copy, `data/sft_filtered_a1_trim`, used by `training_a1_1ep_trim.yaml`.

Reconstructing the untrimmed counterparts of A5/A6/A7 (for the trim contrast): - `scripts/build_untrim_a5a6` rebuilds untrimmed persistent data without re-running the teacher: - **A5-untrim** = the passed A5 records with their trimmed messages replaced by the untrimmed messages from the raw regen output (`data/sft_raw_a5_persistent`), matched by id. The script verifies the filtered (trimmed) messages are an **exact prefix** of the raw (untrimmed) messages — the fidelity guarantee that makes the trim contrast clean. - **A7-untrim** = `convert_a1_to_a7` applied to A1’s *untrimmed* persistent data. - **A6-untrim** = `convert_a5_to_a6` applied to A5-untrim. Output: `data/sft_filtered_a{5,6,7}_untrim/`.

We document these chains explicitly because the untrimmed A5/A6/A7 are *reconstructed*, not trained-from-scratch matched pairs; the exact-prefix verification is the central check that the only thing changing between a trimmed and untrimmed cell is the presence of the post-marker continuation.

4.5 Evaluation

Generations are produced by `scripts/run_paper_eval.py`, one checkpoint at a time, over the frozen held-out probes in `eval_sets/` (20% hash-deterministic seed split, `scripts/build_eval_sets.py`), written to `outputs/paper_v2/eval*/{condition}/{test_set}.jsonl`. Firing is mechanical: a generation “fires” iff it contains a `[SESSION_END...]` marker. The axis, if present, is the token inside `[SESSION_END: <axis>]`.

Split unit and leakage checks. The split is performed at the level of the **source scenario seed id** (a 12-hex content hash of the source scenario), not the individual record: every dialogue and every probe item derived from a scenario carries that scenario’s seed id, so all variants of a scenario are assigned to the same side of the split. This prevents variant-level leakage across training and evaluation. We verify this holds: `scripts/score_leakage_check.py` confirms that of the 224 held-out (eval) scenario ids, **0 appear among the 747 training scenario ids** — a clean seed-level split. The same script finds **0 exact-duplicate and 0 normalized-duplicate** (lowercased, whitespace-collapsed) probe contexts across all 17 evaluation probes, so no eval item is a trivial restatement of another.

Disambiguated trim-study layout. Because the `paper_v2` output dirs encode trim status confusingly (for A5/A6/A7 the *trimmed*-model generations live under `outputs/paper_v2/eval/`, the untrimmed

under `eval_untrim/`; A1’s pair lives under `eval_a1_1ep/` and `eval_a1_1ep_trim/`), we copy them into a self-documenting tree, `outputs/paper_v2/phase0_trim_study/<CELL>_<variant>/`, each dir carrying a `_SOURCE.json` provenance record.

4.6 Scoring scripts

- `scripts/score_sentinel_2x2.py` — recall / FP / premature (paper_v2, mechanical).
- `scripts/score_phase0_attribution.py` — this paper’s F-A/F-B scorer: premature by (design $\times$ marker $\times$ trim) cell, vc-stratified, plus axis-attribution accuracy and the emitted-vs-true axis confusion matrix. Output: `outputs/paper_v2/score/phase0_attribution.json`.
- `scripts/score_mixed_violation_probe.py` — distractor-test scorer for attribution under distraction.

4.7 Probes

- **persistent_probe** — true third-strike positives (recall / attribution).
- **persistent_premature_probe** — single-axis sub-threshold contexts, $n=318$, stratified by (violation_count, premature_turn). Built by `build_eval_sets.py::build_persistent_premature_probe`.
- **mixed_violation_probe** (distractor test) — `scripts/build_mixed_violation_probe.py`. Primary axis X escalates to threshold while distractor axis Y appears sub-threshold; $n \approx 282(141_{fire_correct}/141_{distractor_ub})$, all $12X \times \$Y$ pairs. Generated on the typed checkpoints (A1, A5) and scored by `score_mixed_violation_probe.py` (§5.4; output `outputs/paper_v3/score/mixed_violation.json`).

4.8 Hardware and training configuration

All cells share the configuration below (from `config/paper_v2/training_a1_1ep.yaml` and its per-cell siblings, which differ only in data paths); reported so the training budget the data-shape claim is conditioned on is fully explicit.

Setting	Value
Base model	Qwen3.5 0.8B-Base (vendor/models/Qwen_3.5_0.8B-Base)
Adapter	LoRA/QLoRA (Hu et al. 2022; Detrmers et al. 2023)
LoRA rank / alpha / dropout	16 / 32 / 0.05
LoRA target modules	q,k,v,o,gate,up,down projections
Quantization	4-bit NF4, double-quant, bf16 compute
Epochs	1
Optimizer	paged AdamW 8-bit
Learning rate	2e-4, cosine schedule, warmup ratio 0.05
Effective batch size	8 (per-device $1 \times$ grad-accum 8)
Max sequence length	1792 tokens
Precision / grad checkpointing	bf16 / enabled
Attention impl.	SDPA
Training seed / shuffle seed	42 (primary); 7 (replication)
GPU	single 12 GB consumer card

The off-family replication (§5.5) uses the same recipe on Llama-3.2-1B-Base; its exact config is in Appendix A / `outputs/paper_v3/phase3/`.

4.9 Statistical note

The main design is single training seed (seed 42). We additionally retrain the primary A1 trim/untrim pair at an independent seed (seed 7) and reproduce the trim effect (§5.1); two seeds is a limitation we state explicitly (§6.6) and address with a pre-registered ≥ 3 -seed protocol in future work (§6.6). The trim effect ($\sim +0.5$, §5) is far larger than plausible seed variance and is corroborated by a factorial item-level model (§5.1b); the attribution effect (typed ~ 0.96 vs generic undefined) is structural.

4.10 Ethics, data, and licensing

Ethics. All datasets in this study are *synthetic*: dialogues are generated by a teacher LLM from templated scenarios, contain no human-subject data and no personally identifiable information, and no human participants were involved. The study therefore raises no human-subjects concerns; any journal-specific ethics declaration can be satisfied on this basis.

Licensing. The base models (Qwen3.5, Llama-3.2) are used under their respective model licenses; synthetic data, training/scoring scripts, adapters, and prompts derived in this work are released for research use consistent with those licenses and the project repository’s license. Redistribution of any base-model weights follows the upstream license terms and is not performed here.

Author information. *[To be completed for the non-anonymous submission: author name(s), affiliation or “Independent Researcher,” corresponding-author contact email, and ORCID where available. The current “Independent Research” placeholder should be replaced per the target journal’s requirements.]*

5. Results

5.1 The trim artifact (H1) — primary result

We find that trimming each persistent training record to end at the marker raises premature firing in every (position $\times$ marker) cell, by an amount that dwarfs both design factors. The premature probe is `persistent_premature_probe` ($n=318$ single-axis sub-threshold contexts; firing is the failure).

Cell	position	marker	untrimmed (95% CI)	trimmed (95% CI)	Δ (trim effect)	Fisher p	McNemar p
A1	4-variant	typed	0.207 [0.167, 0.256]	0.783 [0.735, 0.825]	+0.576	4.3e-50	3.5e-49
A5	fixed-7	typed	0.119 [0.088, 0.160]	0.560 [0.505, 0.613]	+0.440	4.0e-33	5.7e-36
A6	fixed-7	generic	0.157 [0.121, 0.201]	0.641 [0.587, 0.692]	+0.484	4.7e-37	8.2e-41
A7	4-variant	generic	0.214 [0.172, 0.262]	0.692 [0.639, 0.740]	+0.478	1.1e-34	3.7e-41

The trim effect is **+0.44 to +0.58**, same sign in all four cells. By comparison the position and marker main effects are $\leq \$0.13$. Trimming is by far the largest observed main effect on premature firing in the tested design (the factorial model of §5.1b makes this comparison formal). (A1’s untrimmed/trimmed pair is the 1-epoch matched retrain, `a1_1ep / a1_1ep_trim`; A5/A6/A7’s untrimmed counterparts are the faithful reconstructions of §4.4.)

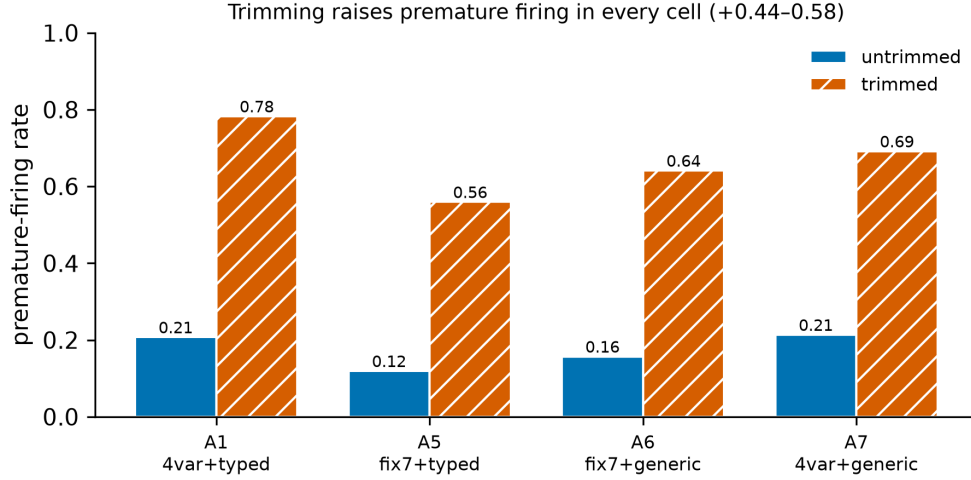


Figure 2: Trimming raises premature firing in every (position×marker) cell; effect sizes +0.44–0.58. Bars are premature-firing rates on the sub-threshold probe (n=318 per cell).

The effect is statistically unambiguous. Wilson 95% confidence intervals for the trimmed and untrimmed rates are disjoint in every cell (n=318 each). The per-cell contrast is significant at Fisher’s exact $p < 10^{-32}$ (two-sided); the paired McNemar exact test — comparing trim and untrim decisions on the *identical* probe items — gives $p < 10^{-35}$ in all four cells (Appendix A). The effect sizes are large by conventional standards across all four cells: odds ratio 8.2–13.6, Cohen’s h 0.98–1.23 (risk difference = the trim Δ above).

5.1b Factorial analysis — trim is the largest main effect

The per-cell tests above establish the trim contrast within each design cell; to support the stronger statement that trim *dominates* the position and marker factors, we analyse all eight cells jointly as the fully crossed $2 \times 2 \times 2$ design they form. We fit an item-level logistic model of premature firing on the three binary factors and their interactions,

$$\text{logit } P(\text{fire}) = \beta_0 + \beta_1 T + \beta_2 P + \beta_3 M + \beta_4 TP + \beta_5 TM + \beta_6 PM + \beta_7 TPM,$$

with T = trim (1/0), P = position (4-variant/fixed-7), M = marker (typed/generic). Because the same probe items recur across cells, we report cluster-robust (sandwich) standard errors clustered on probe item id (n=2544 item-level observations, 318 item clusters).

Term	β (log-odds)	Odds ratio	Cluster SE	z	p
trim (T)	+2.26	9.59	0.171	+13.2	5.6e-40
position (P)	+0.38	1.46	0.165	+2.29	0.022
marker (M)	-0.32	0.73	0.180	-1.76	0.078
$T:P$	-0.15	0.86	0.214	-0.70	0.483
$T:M$	-0.02	0.98	0.226	-0.10	0.917
$P:M$	+0.28	1.32	0.234	+1.20	0.231
$T:P:M$	+0.54	1.71	0.311	+1.72	0.085

Trim is by far the largest term (odds ratio 9.6, $p \approx 5.6 \times 10^{-40}$); position is a small positive effect, marker is not significant, and **no interaction is significant** — i.e. the trim effect is additive and consistent across position and marker format, matching its same-sign appearance in all four cells. Converting to the probability scale, the average marginal (model-adjusted) main effect of each factor on $P(\text{premature fire})$ is **trim +0.494**, position +0.105, and marker -0.009. This quantifies the “largest observed main effect” claim of §5.1 without relying on cross-cell eyeballing. Scored by `scripts/score_phase0_factorial.py`.

Both seed-42 A1 models clear the positive-probe recall gate on true third-strikes: recall **0.893** (untrimmed) / **0.994** (trimmed) on `persistent_probe` ($n=159$, greedy decoding). So the premature contrast is between two models that have both learned to fire on genuine third-strikes, not an artifact of one model failing to fire at all; if anything the trimmed model fires *more* readily on true positives too, consistent with its inflated firing tendency overall.

Seed replication (two seeds; see limitations). Retraining the A1 trim/untrim pair at an independent seed (seed 7) reproduces the effect: premature firing rose from 0.104 to 0.620, $\text{trim}\Delta = +0.516$, the same direction and magnitude as the seed-42 primary result of +0.576. Recall was 0.81 untrimmed and 0.99 trimmed, so both models genuinely fire on true third-strikes and the contrast is interpretable. We are transparent about the checkpoint choice: the seed-7 pair is reported at its earliest shared checkpoint that clears the recall gate for *both* variants (checkpoint-600, epoch 1.51), rather than the exact 1-epoch mark used for the seed-42 primary result, because at exactly 1 epoch the seed-7 *untrimmed* model had not yet crossed the recall gate — evaluating a premature-firing contrast on a model that has not learned to fire at all would be uninterpretable. The reseed changes only the reported checkpoint, not the data or hyperparameters. We report two seeds rather than the three or more a full variance characterization would use; given a per-cell effect of +0.44–0.58 at Fisher $p < 10^{-32}$ with disjoint Wilson intervals, and the same direction at an independent seed and an off-family replication (§5.5), sampling variance is an implausible explanation for the effect, but we flag the seed count and checkpoint-selection choice as limitations (§6.5) and specify a pre-registered ≥ 3 -seed protocol as future work (§6.6).

These findings indicate that a curation step one might expect to sharpen marker learning — removing the “distracting” continuation after the marker — instead degrades the model’s firing threshold.

5.2 The effect tracks strike count, over and above turn position (H1 support)

We find that premature firing tracks accumulated violation count, and that this holds after adjusting for turn depth. Stratifying the premature probe by violation count (vc = number of prior same-axis strikes, both sub-threshold; full per-cell table in Appendix B, Table B.1) shows two facts. (i) In every trimmed cell, firing jumps sharply from $vc=1$ to $vc=2$ (e.g. A1 trimmed $0.654 \rightarrow 0.912$) — the model fires *more* the more escalation it has seen, rather than waiting for exactly the third strike. (ii) Untrimmed cells stay low at *both* vc levels (A1 untrimmed $0.063 \rightarrow 0.352$). This is a threshold-laxity signature on the strike *count*. Because count and depth are correlated in natural dialogue, §5.2b isolates them; we defer the mechanistic account — why the effect sits on the escalation feature rather than raw depth, and its direct logit-level signature — to §6.1.

5.2b Strike count survives adjustment for turn depth (confound check)

A natural concern is that violation count and turn depth are correlated in dialogue, so §5.2 could reflect raw depth. The premature probe is not a fully balanced depth $\times$ count grid, but three depths (turns 3,

5, 7) carry *both* strike counts, so within each of these depths we can read the pure effect of adding a strike while holding depth fixed. At every such depth and in every cell, adding the second strike raises firing (e.g. A1 untrimmed at fixed depth: +0.28 at turn 3, +0.22 at turn 5, +0.12 at turn 7; A1 trimmed: +0.38 / +0.29 / +0.20), so the count effect is not an artifact of deeper contexts (Figure 5).

At fixed turn-depth, adding a strike raises firing — count, not depth (§5.2b)

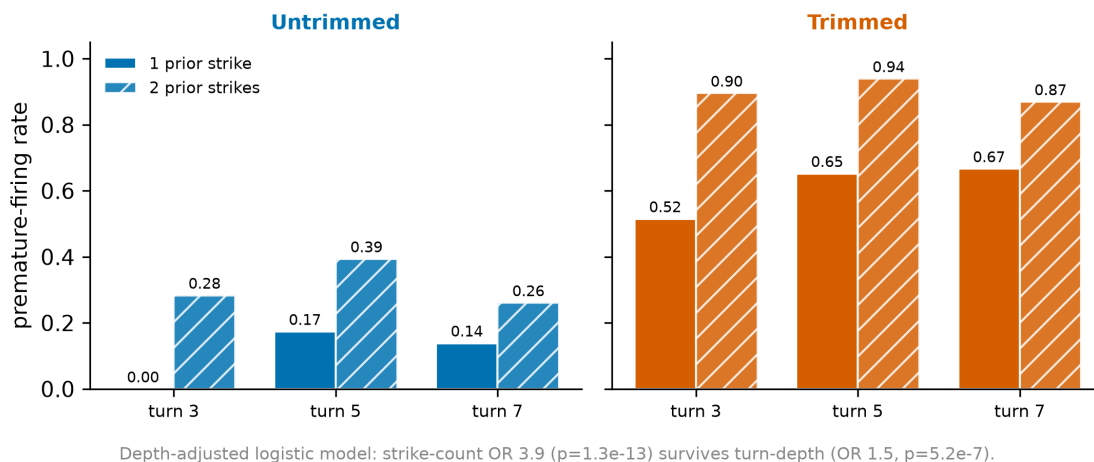


Figure 3: Confound check (§5.2b). Holding turn-depth fixed at each of the three depths that carry both strike counts (turns 3, 5, 7), adding the second sub-threshold strike (hatched bars) raises premature firing over one strike (solid) in both the untrimmed and trimmed A1 models — so firing tracks accumulated strike count, not raw turn depth. A depth-adjusted item-level logistic model confirms the strike-count term survives (OR 3.9, $p=1.3e-13$).

We confirm this with an item-level logistic model on the pooled typed cells (A1+A5, both variants; $n=1272$ observations over 318 item clusters, cluster-robust SE):

term	β (log-odds)	odds ratio	z	p
turn depth (standardized)	+0.43	1.54	+5.02	5.2e-7
strike count (vc=2 vs 1)	+1.37	3.92	+7.41	1.3e-13
trim (vs untrim)	+2.78	16.1	+18.35	3.2e-75

The strike-count term remains strongly positive and highly significant *after* adjusting for depth (and trim), while depth carries a smaller independent effect. The most direct interpretation supported by this evidence is that accumulated strike count — not raw turn position — is the feature premature firing rides on. A fully orthogonal, balanced depth $\times$ strike-count probe (matched cells across several depths at both sub-threshold counts) would test this more directly and is flagged as future work (§6.6). Scored by `scripts/score_depth_count_grid.py`.

5.3 Typed markers are semantic gates (H2)

We find that when a typed model fires, it names the *correct* axis almost always, and it never emits a contentless marker. A generic model cannot attribute at all — by construction its marker carries no axis.

Axis-attribution accuracy = of fires carrying an axis label, the fraction naming the true violated axis. Scored on `persistent_probe` (true positives) and on the premature probe’s fires.

Cell	marker	variant	attribution acc. (probe)	contentless fires
A1	typed	untrimmed	0.985	0
A1	typed	trimmed	0.962	0
A5	typed	untrimmed	0.966	0
A5	typed	trimmed	0.938	0
A6	generic	untrimmed	— (undefined)	all fires
A6	generic	trimmed	— (undefined)	all fires
A7	generic	untrimmed	— (undefined)	all fires
A7	generic	trimmed	— (undefined)	all fires

Typed models attribute at **0.94–0.99** and emit zero bare markers; generic models produce only contentless fires. Crucially, attribution is **robust to trim** (A5: 0.966 $\rightarrow$ 0.938) even where trim substantially worsens *timing* (A5 premature 0.119 $\rightarrow$ 0.560). So *when* a model fires and *why it says it fires* are separable, separately-curated behaviors: trim damages timing but not attribution; marker typing governs attribution independent of timing. This is the answer to “what is the typed marker for” — not lower premature firing, but an accurate, actionable classification of the violated invariant.

5.3b A fuller control-token reliability taxonomy

Recall, premature firing, and attribution are the failure modes central to our claims, but deployment reliability also depends on benign false positives, misses, off-position (late) firing, malformed markers, and duplicate emissions. We report the full operational set below, re-scored from the existing generations (`scripts/score_reliability_taxonomy.py`). Two probes — the fully benign false-positive probe (`persistent_fp_probe`) and the off-position probe (`persistent_offposition_probe`) — were generated for the trim variants only, so those columns are reported where available.

Cell	Variant	Recall	Miss	Benign FPR	Off-pos recall	Malformed	Dup-fire	E2E (fire $\wedge$ axis)
A1	untrim	0.818	0.182	—	—	0.000	0.000	0.805
A1	trim	0.994	0.006	—	—	0.000	0.000	0.956
A5	untrim	0.560	0.440	—	—	0.000	0.000	0.541
A5	trim	0.912	0.088	0.000	0.797	0.000	0.000	0.855
A6	untrim	0.679	0.321	—	—	0.000	0.000	—
A6	trim	0.912	0.088	0.004	0.960	0.000	0.000	—
A7	untrim	0.755	0.245	—	—	0.000	0.000	—
A7	trim	0.987	0.013	0.004	0.946	0.000	0.000	—

Two facts matter for interpreting the premature result. First, the marker syntax is reliable: the **malformed-marker and duplicate-fire rates are 0.000 in every cell** — the models emit well-formed, single markers, so premature firing is a timing failure, not a formatting one. Second, the **benign false-positive rate is $\approx$ \$0.000–0.004**: the trimmed models almost never fire in fully benign contexts. Premature firing is therefore specifically a *sub-threshold escalation* laxity (firing when a governed axis is escalating but has not yet reached threshold), not indiscriminate over-firing. The end-to-end column reports $P(\text{fires} \wedge \text{names the correct axis})$ on the positive probe — the deployment-relevant joint event — which for the typed cells reaches 0.956 (A1 trim) and complements the conditional attribution of §5.3.

5.4 Attribution under distraction — the mixed-axis distractor condition (H2)

The §5.3 attribution is measured on single-axis conversations, where naming the axis is comparatively easy. The *mixed_violation_probe* (§4.7) is the stronger test: primary axis X escalates to threshold while a distractor axis Y appears sub-threshold. A model that supports genuine per-axis attribution fires naming X, not Y, and does not fire merely because *some* axis is escalating.

Attribution is a typed-only metric — a generic marker carries no axis, so A6/A7 are not scorable here and are omitted. Scored over the two typed cells (n=141 *fire_correct* records each; full table in Appendix B, Table B.2).

The semantic gate holds under distraction: when a typed model fires with a distractor axis Y present in the conversation, it names the *threshold* axis X on **0.975 / 0.963** of fires and is pulled to the distractor on under 2% (wrong-axis 0.009 / 0.018), never emitting a contentless marker. These rates are statistically indistinguishable from the single-axis attribution of §5.3 (0.94–0.99) — the distractor barely dents them. So typed markers support robust *per-axis* attribution even under a competing sub-threshold distractor, rather than a generic “something is wrong” reflex: this is the strongest form of the semantic-gate evidence, because unlike the typed-vs-generic contrast (which is attribution-capable by construction) it requires the model to select the *threshold* axis over a competing one.

Secondary: the *distractor_sub* records (X at 2 strikes, Y present, n=141) measure whether a distractor axis inflates *premature* firing. The untrimmed typed models fire prematurely at **0.567 (A1) / 0.425 (A5)** here — roughly 2–3× their single-axis untrimmed premature rate (0.207 / 0.119, §5.1). Adding a second escalating axis to the context amplifies threshold-laxity, consistent with the escalation-driven mechanism (§6.1): more accumulated escalation pressure, more premature firing. This extends the §5.1 story to a two-axis context; it is not central to the semantic-gate claim.

5.5 Generalization — off-tutor replication (H-gen)

To show the trim→premature effect is a property of *next-token training on rare, count-triggered semantic markers* and not an artifact of the tutoring corpus, we replicate it in a different domain on a different model family (Appendix A). The trigger must remain semantic and recognition-gated — a literal token-counting task would let the model count exactly, leaving no recognition noise for the trim to exploit (§6.1) — so we use synthetic customer-support dialogues: the agent must emit [ESCALATE] after the 3rd explicit escalation request (“let me speak to a manager / a human”). The counted trigger is the *request*, so the label matches the content a reader would count; angry-but-non-requesting venting is a distractor that must not count, making per-message recognition a genuine semantic judgment rather than a keyword match. Requests are interleaved with 0–2 distractor exchanges at random positions so the trigger is decorrelated from turn index. We fine-tune **Llama-3.2-1B-Base** (a non-Qwen, full-attention base — so this run *also* speaks to the cross-family magnitude discussion of §6.6) on trimmed (dialogue ends at [ESCALATE]) vs untrimmed (benign resolution turns follow) data. The two corpora are generated from a single shared draw and differ *only* by the post- [ESCALATE] continuation (verified: the trimmed dialogue is an exact prefix of its untrimmed counterpart for all 3000 records), so the trim is the sole manipulated variable. Both are trained with identical hyperparameters and seed, with loss masked to assistant turns only (matching the tutor SFT). Premature emission is measured on held-out sub-threshold contexts (1–2 requests), with a positive-probe (3 requests) recall gate ensuring both models learned the task.

The trim effect replicates in direction (Table below). Both variants reach perfect recall on the positive probe (1.00), so the premature comparison is valid. The trimmed Llama model fires [ESCALATE]

prematurely on sub-threshold contexts more than the untrimmed one:

task / model	premature (untrimmed)	premature (trimmed)	trim Δ	recall gate
tutor, Qwen 0.8B (A1, §5.1)	0.207	0.783	+0.576	—
support-chat, Llama-3.2-1B-Base	0.683	0.830	+0.147	1.00 / 1.00

Table: Off-tutor, off-family replication of the trim→premature effect. Numbers are the first-epoch checkpoint (both models pass the recall gate at 1.00); the trimmed model emits the escalation marker on sub-threshold contexts more often than the untrimmed model, the same direction as the Qwen tutor result.

The central claim is *direction*, not magnitude: the trimmed Llama model emits [ESCALATE] prematurely more than the untrimmed one — in a different domain, on a different family, with no tutoring or Qwen lineage in the loop — so the effect replicates in direction on a second synthetic task and a different model family, suggesting it is not specific to the tutoring corpus or the Qwen family, and that the curation principle (§6.3) plausibly transfers to other rare, count-triggered semantic control markers. (Consistent with §5.1’s primary result, the elevated *absolute* premature rate on this synthetic task — both variants are high because the sub-threshold probe is deliberately adversarial — makes the trimmed-vs-untrimmed *gap* the meaningful quantity.)

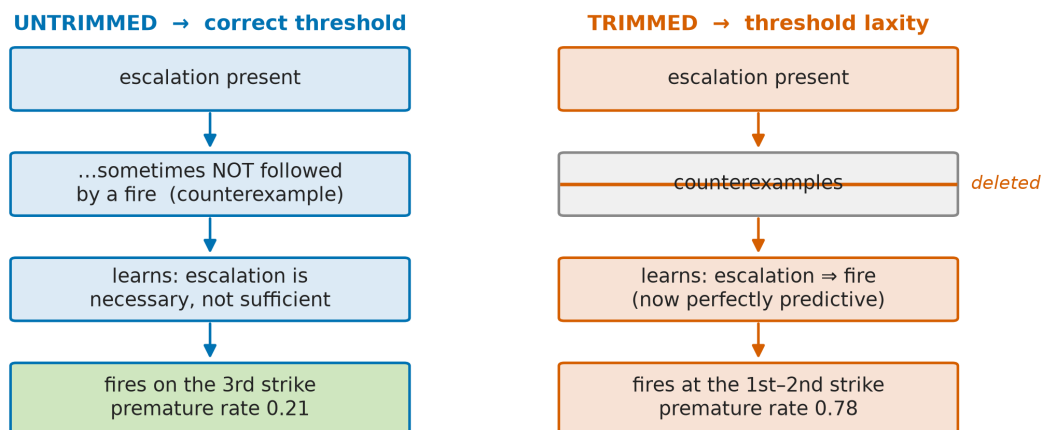
5.6 Summary

- **H1 [supported]:** trimming → premature firing, +0.44–0.58 across all four cells; the largest observed main effect in the tested $2 \times 2 \times 2$ design (item-level factorial logistic model, §5.1b: trim odds ratio ≈ 9.6 , all interactions n.s.).
- **H1 mechanism [evidence-consistent]:** most consistent with threshold-laxity on accumulated violation count rather than a turn-position shortcut. Evident at the logit level: the trimmed model puts $34\text{--}45\times$ more probability mass on beginning the sentinel at sub-threshold escalation than the untrimmed model; and the strike-count term survives adjustment for turn depth (§5.2b, §6.1).
- **H2 [supported, single-axis]:** typed = semantic gate; 0.94–0.99 attribution, 0 contentless fires; generic cannot attribute. Attribution is trim-robust.
- **H2b [supported]:** attribution holds under a sub-threshold distractor — correct-axis 0.975 (A1) / 0.963 (A5), wrong-axis < 0.02 , 0 contentless fires, matching the single-axis rate. The strongest semantic-gate evidence, since it requires selecting the threshold axis over a competing one.
- **H-gen [supported in direction]:** the trim→premature effect replicates off-tutor and off-family. On a synthetic customer-support escalation task (fire after the 3rd *explicit* escalation request), fine-tuning Llama-3.2-1B-Base on trimmed vs untrimmed data (identical shared draw, differing only by the post-marker continuation) yields a trimmed model that fires the escalation marker prematurely more than the untrimmed one (0.830 vs 0.683, both at recall 1.00) — the same direction as the Qwen tutor result. This suggests trim→premature is a data-shape effect not specific to the tutoring corpus or the Qwen family; we report direction, not magnitude, given the small synthetic setting.
- **Candidate remedy [future work]:** supervising the strike count directly in the marker is a natural fix for the trim artifact; we outline it as future work (§6.6) rather than a tested contribution.

6. Discussion

6.1 Why trimming causes premature firing (the mechanism we find most consistent with the evidence)

Why trimming causes premature firing



Trimming removes the post-marker turns that show escalation NOT followed by a fire, leaving the trigger perfectly predictive.

Figure 4: The mechanism. Untrimmed data contains counterexamples — escalated contexts *not* followed by a fire — so the model learns escalation is necessary but not sufficient and fires on the third strike. Trimming deletes those counterexamples, leaving escalation perfectly predictive of the marker, so the model fires early.

The naive story — “trimming teaches the model the marker sits at the end of the sequence, so it fires when it detects the end” — is not compatible with a literal future-aware explanation: an autoregressive, strictly causal decoder generating token t conditions only on tokens $1..t-1$ and has no access to whether more tokens will follow, so “am I at the end?” is not a feature it can compute at generation time. (This argument is about causality and so applies to any strictly causal decoder — linear-attention or gated-recurrent variants included — though we test the effect empirically on two families, not all architectures.) We note that present-position and other contextual termination cues remain *possible* alternative correlates; the account we advance below is the one most directly supported by the present evidence, not the only conceivable one.

Our evidence is most consistent with an account about which *counterexamples* trimming removes — a shortcut-learning story (Geirhos et al. 2020; McCoy et al. 2019) in which the cheapest sufficient predictor wins, framed as the best-supported explanation rather than a proven mechanism. SFT optimizes $P(\text{marker} \mid \text{left-context})$. The marker’s true trigger is a *count* (third same-axis strike); the cheap left-visible correlate is *escalation-presence*. In untrimmed records, the turns *after* the marker show an escalated context **not** followed by a fire — direct supervision that escalation is necessary but not *sufficient* — and trimming deletes exactly those turns, so the model fits $P(\text{fire} \mid \text{escalation-present}) \approx 1$ and fires after one or two strikes rather than counting to three. This is precisely the $\text{vc}=1 \rightarrow \text{vc}=2$ rise, depth-adjusted, that §5.2/§5.2b document.

Semantic recognition is a precondition, not incidental. The shortcut is available only because the model’s *count* of same-axis strikes is uncertain, and it is uncertain because each strike must first be *recognized* — “is this user turn a role-swap attempt?” is a fuzzy semantic judgment, not a token match. The trim does not teach “fire on a marker token seen at the end”; it teaches “fire once the *noisy accumulated evidence* of the semantic trigger is high,” because trimming removed the examples showing that high evidence short of the exact threshold is not yet a fire. This is why the effect requires a semantic, recognition-gated trigger: a task that counts an *explicit, unambiguous* token would let the model count exactly and leave no recognition noise for the trim to exploit — no shortcut, no premature firing. The generalization test (§5.5) is therefore constructed on a *different-domain but still semantic* trigger (escalation after repeated *angry* customer messages, where “angry” is recognition-gated), not on literal token counting, precisely because the recognition noise is the substrate the trim shortcut operates on.

The logit-level signature. The evidence supports this account directly at the logit level: for each sub-threshold context we read the probability the model *begins the sentinel* — the joint $P([\] \times P(\text{SESSION} \mid [\])$, a two-token teacher-forced measurement (not sampling; Appendix A) that isolates the sentinel from any other bracketed token — for the A1 trim/untrim pair over the same 318 sub-threshold contexts, stratified by accumulated escalation (violation count):

violation count	untrimmed P(sentinel)	trimmed P(sentinel)	ratio
1 (one prior strike)	0.000005	0.000213	45×
2 (two prior strikes)	0.000045	0.001530	34×

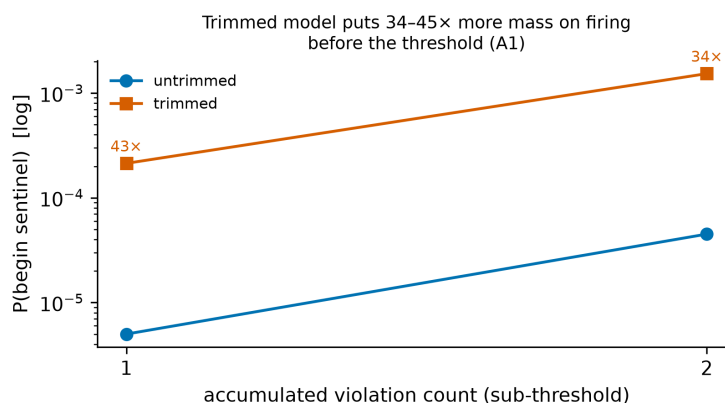


Figure 5: Logit-level signature (A1). At sub-threshold escalation the trimmed model places 34–45× more probability mass on beginning the sentinel than the untrimmed model, and that mass grows with accumulated violation count (log scale).

The graded internal signal supports the prose mechanism directly: trimming inflates the model’s conditional probability of firing given escalation, well before the threshold (untrimmed mass stays near-zero until the true third strike; the trimmed model’s is pushed toward the boundary at every sub-threshold level). And this sits on the escalation *feature*, not raw depth: the shared $\sim 4/5$ benign majority already decorrelates depth from firing, and holding depth fixed the strike-count term survives adjustment (§5.2b). The most direct interpretation is that escalation count, not raw depth, drives the effect — though a fully orthogonal balanced probe would test it more directly (§6.6).

6.2 Timing and attribution are separable

Trim substantially worsens *timing* (premature firing) but leaves *attribution* almost untouched (A5 attribution 0.966 $\rightarrow$ 0.938 while premature 0.119 $\rightarrow$ 0.560). Marker typing governs attribution independent of timing. So “a control marker” is really two learned behaviors — a trigger (when) and a classifier (why) — with independent curation levers: the trim (or post-marker continuation) governs the trigger threshold; the marker’s semantic content governs the classifier. A practitioner can get one right and the other wrong; both must be curated.

6.3 A curation principle for rare control tokens

Generalizing beyond this task: **when training a model to emit a rare marker on a threshold/count trigger, do not trim training sequences to end at the marker.** Trimming maximizes the correlation between the marker and sequence-terminality/ trigger-presence, deleting the negative examples the model needs to learn that the trigger’s *presence* is not its *threshold*. Retain a benign continuation after the marker. Where the trigger is a count, additionally supervise the count explicitly in the output (§6.6). We conjecture — but do not demonstrate — that this principle extends to other rare, machine-consumed markers whose trigger is a *noisy, recognition-gated* count or threshold: refusal-after-N, escalate-after-repetition, stop-after-goal. We deliberately scope the claim this way. The mechanism (§6.1) requires that the count be *latent and uncertain* for trimming to have counterexamples worth deleting; where the trigger is exactly computable from surface tokens, there is no noise to exploit and we predict no effect (a literal token-count trigger; see the mechanism argument in §6.1). We do not run this control experiment; it follows from the mechanism rather than being established here. Whether the artifact appears for markers with crisp, non-semantic triggers — e.g. schema-driven tool/function-call or JSON control tokens emitted on an exact syntactic condition — is therefore an open empirical question we do not settle here.

6.4 Practical recommendations

The findings translate into concrete, low-cost guidance for anyone building SFT corpora that contain rare machine-consumed markers:

1. **Retain a benign continuation after the marker.** Do not trim training sequences to end at the marker; keep the post-marker turns that show an escalated context *not* followed by another fire (§5.1, §6.1). This is the highest-leverage curation choice we identify, and substantially reduces premature firing.
2. **Treat sequence shaping as a documented experimental variable**, not a silent preprocessing detail — where a sequence is truncated can move premature firing by +0.5 (§5.1).
3. **Evaluate timing separately from semantics.** Conventional fluency/quality metrics do not detect premature firing; a dedicated sub-threshold probe does (§5.2). *When* and *why* a marker fires are separately curated behaviors (§6.2) and must be measured separately.
4. **Where the trigger is a count, supervise the count** (§6.6) rather than leaving it a latent variable the trim can corrupt.

These cost little beyond corpus bookkeeping. We demonstrate them at a single small scale (0.8B) and on two model families; whether the magnitude holds at much larger scale is left to future work (§6.5, §6.6).

6.5 Threats to validity

Scope of the claims. We claim, and support with a controlled paired experiment, a specific effect: for a rare marker whose trigger is a *noisy, recognition-gated* count, trimming training sequences to end at the marker inflates premature firing. Because the trimmed and untrimmed corpora are byte-identical except for the post-marker continuation (§4.4, and the exact removed-tail accounting of §4.5), the *trim manipulation* is a clean causal intervention. The *explanation* we offer for it — deletion of the sub-threshold counterexamples inducing threshold-laxity (§6.1) — is the account most consistent with the evidence (including the logit-level signature and the depth-adjusted strike-count model), not a proven mechanism to the exclusion of all contextual alternatives. We do *not* claim this is a universal property of all control tokens, all curation pipelines, or all model scales. In particular the effect is demonstrated at 0.8B (Qwen3.5) plus an off-family 1B base (Llama-3.2-1B-Base, §5.5), on count/threshold triggers, and with SFT; extrapolation to much larger models, to exactly-computable triggers (§6.3), or to RL/preference post-training is conjecture. The remaining bullets enumerate the specific axes along which our evidence is thin.

- **Single family for the main 2\$×2×2 — a *secondary* threat here.** All eight cells use Qwen3.5 0.8B, but our claims are about the *training signal* (which counterexamples trimming deletes), not a specific model’s capability, and the off-family, off-domain Llama-3.2-1B-Base replication (§5.5) already shows the effect’s *existence* is family- and domain-independent. What it does not settle is *magnitude*: the Qwen3.5 0.8B base is a hybrid-attention architecture (per its released config, most layers are linear-attention with a full-attention layer at a fixed interval), and its compressed cross-turn state could plausibly amplify trim sensitivity relative to a fully full-attention model. A matched A1 trim/untrim contrast on a small full-attention non-Qwen base would test this; it is the open cross-family *magnitude* question (§6.6). We flag this as a hypothesis about magnitude only — the effect’s existence does not depend on it.
- **Reconstructed untrimmed cells.** The untrimmed A5/A6/A7 are rebuilt by reconstruction (Appendix A), not trained-from-scratch matched pairs. The exact-prefix verification (§4.4) is the guarantee that only the post-marker continuation differs; still, an independent from-scratch re-generation would be stronger.
- **Seed count.** The main eight-cell design is trained at a single seed (42), with the primary A1 trim/untrim pair additionally replicated at seed 7 — two seeds in total for the primary contrast. Two seeds do not fully characterize training variance. We rely instead on effect *magnitude* (per-cell +0.44–0.58, Fisher $p < 10^{-32}$, disjoint Wilson intervals) plus same-direction reproduction at an independent seed and an off-family model to argue the result is not seed-driven; a pre-registered ≥ 3 -seed run at fixed checkpoints (seeds 7/42/123, five if budget permits) remains the proper test and is specified in §6.6.
- **Checkpoint selection for the seed-7 pair.** The seed-7 pair is reported at the earliest shared checkpoint clearing the recall gate for *both* variants (checkpoint-600, epoch 1.51), not the exact 1-epoch budget of the seed-42 primary, because at 1 epoch the seed-7 untrimmed model had not yet learned to fire and a premature-firing contrast on a non-firing model is uninterpretable. This is a principled, pre-specified gate (recall $\geq$ threshold for both variants), not an outcome-maximizing search over checkpoints, but we flag it because it could be read as checkpoint selection; the fixed-checkpoint multi-seed protocol (§6.6) removes the ambiguity.
- **Token-budget confound (bounded, not eliminated).** Trimming removes the post-marker continuation, which also removes a small number of supervised tokens (§4.5: mean $\approx$ \$14 assistant tokens, exactly one turn, per persistent dialogue). In principle the effect could ride on “fewer supervised tokens” rather than “the missing counterexamples” specifically. Two facts bound this:

the removed tokens are $\approx 1\%$ of the persistent-stream supervised tokens and a far smaller fraction of the whole corpus, and the depth-adjusted count model (§5.2b) locates the effect on the escalation feature rather than on generic sequence length. A definitive separation is a *token-matched control* — adding the same number of extra supervised tokens as benign continuation *without* the high-escalation/no-fire structure — which we specify in §6.6.

- **Single dataset / task — addressed by the off-family replication.** The trim principle is demonstrated primarily on the tutoring persistence task; the §5.5 Llama replication (a different domain and family) is what extends it beyond this corpus, keeping the trigger semantic and recognition-gated because recognition noise is the substrate the shortcut requires (§6.1).
- **Synthetic data — a methodological necessity, not merely a convenience.** Both datasets are synthetic. Our method is a *single-variable paired contrast* — a trimmed and untrimmed corpus byte-identical except for the post-marker continuation (exact prefix, §4.4) — and no pre-existing real dataset supplies such a pair; constructing one requires regenerating real dialogues with and without the continuation, at which point the controlled corpus is synthetic by construction. Real corpora also lack the labelled sub-threshold probes that make premature firing measurable. Synthetic control is thus what lets us attribute the effect to the trim alone; a naturalistic study (auditing an existing tool-calling corpus for trim-correlated premature calls) remains valuable future work.

6.6 Future work

The most informative next experiments, in priority order (the first three most directly strengthen the causal claim and are the ones we would run before any broadening of scope):

1. **Multi-seed at fixed checkpoints.** Train the primary A1 trim/untrim pair at ≥ 3 pre-registered seeds (7, 42, 123; five if budget permits), evaluate all at *identical* fixed checkpoints, and report mean $\pm$ SD with learning curves for both recall and premature firing. This replaces the current two-seed result and removes the checkpoint-selection ambiguity (§6.5).
2. **Counterexample dose-response and token-matched control.** Vary the *retained* post-marker continuation across C0/C25/C50/C75/C100 (0/25/50/75/100% of the tail kept) and plot premature firing against retained counterexample supply, to show the effect is continuous in counterexample dose rather than a binary switch. In addition, a *token-matched* control adds the same number of extra supervised tokens as C100 but as benign continuation *without* the high-escalation/no-fire structure; if only the genuine counterexample-bearing tail reduces premature firing, the mechanism (§6.1) is established rather than merely evidence-consistent, and the token-budget confound (§6.5) is closed.
3. **Orthogonal depth $\times$ strike-count probe.** Build a balanced grid that independently varies turn depth (e.g. 5/7/9/11) and strike count (1/2) at fixed semantic axis, with matched cell sizes, and estimate $P(\text{fire}) = f(\text{count}, \text{depth}, \text{trim})$. §5.2b already shows count survives depth on the *existing* (unbalanced) probe; a purpose-built balanced grid would settle the depth-vs-count question directly.
4. **Cross-family magnitude** — re-run the A1 contrast on a small full-attention non-Qwen base (Llama-3.2-1B / SmolLM2-360M) to test whether the +0.5 magnitude is specific to Qwen3.5’s hybrid (mostly linear) attention; direction is already family-independent via the off-family replication (§5.5).
5. **Scale** — repeat the contrast at 4B (and larger) to test whether the effect size is scale-dependent; the present study fixes scale and does not claim scale independence.
6. **Count-annotated marker as a remedy.** A natural fix for the trim artifact is to stop keeping

the strike counter latent: tag every strike with its running count ([STRIKE=2: axis] ... [SESSION_END: STRIKE=3: axis]), so premature firing would require emitting a wrong count that the loss penalizes — process supervision (Nye et al. 2021; Wei et al. 2022; Lightman et al. 2024) compressed to a single tag, at negligible inference cost. An exploratory run at 0.8B under-fired the terminal marker (positive-probe recall below the gate), so we leave a recall-cleared evaluation — with a larger context window and a count marker that drops the axis label to isolate the count — to future work.

7. Conclusion

We studied how the *shape* of SFT data governs a fine-tuned LM’s emission of a rare, machine-consumed control marker, using a four-axis session-ending sentinel as a controlled testbed.

Our primary result is counter-intuitive and robust: **trimming training sequences to end at the marker — a curation step one might expect to sharpen marker learning — instead induces premature firing.** Across every (position $\times$ marker) cell, trimming raised the premature-firing rate by +0.44 to +0.58, the largest observed main effect in the tested design. The evidence is most consistent with a threshold-laxity mechanism, rather than a turn-position shortcut or a literal “detects the end” account (which is not available to a strictly causal decoder): trimming deletes the training examples in which an escalated context is *not* followed by a fire, leaving escalation-presence nearly always predictive of the marker and impairing the model’s ability to threshold on the strike count. Retaining the benign post-marker continuation substantially reduces premature firing.

We further showed that *when* a model fires and *why it says it fires* are separable, separately-curated behaviors: **typed markers act as semantic gates**, yielding 0.94–0.99 correct-axis attribution and zero contentless fires, where generic markers cannot attribute at all — and attribution is robust to the trim that worsens timing. Typed markers support robust per-axis attribution under a sub-threshold distractor (correct-axis 0.96–0.98, pulled to the distractor <2%). Finally, the trim effect replicates *in direction* off-family and off-domain — fine-tuning Llama-3.2-1B-Base on a synthetic customer-support escalation task reproduces trim→premature (trimmed 0.830 vs untrimmed 0.683, both at full recall) — suggesting it is not specific to the tutoring corpus or the Qwen family. We also outline **count-annotated markers** — supervising the strike count directly in the marker — as a candidate remedy for the trim artifact, and leave its evaluation to future work (§6.6).

The practical takeaway is a curation principle for any rare control token with a count/threshold trigger: **do not trim to the marker; retain a continuation after it; and where the trigger is a count, supervise the count explicitly.** These are cheap, low-overhead data choices with first-order effects on control-token reliability, observed across two model families (Qwen3.5, Llama-3.2).

Appendix A. Reproducibility

Every result in the paper is produced by a named script over generations already on disk (no result depends on unseen inference). Base model: Qwen3.5-0.8B-Base, QLoRA SFT, 1-epoch matched budget unless noted; the off-family replication (§5.5) uses Llama-3.2-1B-Base. The table below maps each reported result to the script that computes it and the JSON it writes.

Result (section)	Script	Output
Trim × (position×marker) premature rates (§5.1)	scripts/score_phase0_attribution.py	outputs/paper_v2/score/phase0_attribution
A1 positive-probe recall gate, 1-epoch (§5.1)	scripts/run_paper_eval.py --baseline v3_a1_{untrim,trim} --test-set persistent_probe	outputs/paper_v3/eval_recall_check/v3_a1
Significance: Fisher / McNemar / Wilson CIs + effect sizes (odds ratio, Cohen's <i>h</i>) (§5.1)	scripts/score_phase0_significance.py (stdlib only)	outputs/paper_v2/score/phase0_significance
Seed-7 replication (§5.1)	scripts/run_multiseed_s7.py → scripts/score_multiseed_s7.py	outputs/paper_v2/score/phase0_multiseed_s
vc-stratified premature (§5.2)	scripts/score_phase0_attribution.py	outputs/paper_v2/score/phase0_attribution
Logit-level P(sentinel) vs depth (§6.1)	scripts/score_sentinel_logprob_outlets.py	outputs/paper_v3/score/sentinel_logprob_v
Single-axis typed attribution / semantic gate (§5.3)	scripts/score_phase0_attribution.py	outputs/paper_v2/score/phase0_attribution
Attribution under distraction / mixed-violation probe (§5.4)	scripts/score_mixed_violation_probes.py	outputs/paper_v3/score/mixed_violation.js
Count-marker (Design B) construction — future-work remedy (§6.6)	scripts/convert_typed_to_count.py scripts/assemble_a8_count.py	data/sft_filtered_a8_count*
Off-family Llama replication (§5.5)	scripts/phase3_synthetic_trim.py	outputs/paper_v3/phase3/phase3_result.js
Untrimmed A5/A6/A7 reconstruction (§4.4, §6.5)	scripts/build_untrim_a5a6a7.py	outputs/paper_v2/phase0_trim_study/

The firing detector used throughout is the axis-capturing regex `\[SESSION_END(?:\s*:\s*(?:STRIKE=\d+\s*:\s*))` which matches generic, typed, and count-annotated marker forms and captures the axis label for attribution. Code, configs, and the synthetic datasets are in the released repository (see Code and Data Availability); model weights and large training outputs are excluded.

Code and Data Availability. All code, per-condition training configurations, seeds, frozen evaluation sets, judge prompts, the synthetic datasets, and the score outputs underlying every table are released at <https://github.com/cch-ai922/tutor-train>. A reproducibility/ guide maps each result to the script, config, and expected number that produce it. The off-family replication (§5.5) uses Llama-3.2-1B-Base; trained LoRA adapters are low-rank deltas over the public base models and are available from the authors on request.

Appendix B. Supporting tables

Table B.1 — Premature firing stratified by violation count (§5.2). vc = number of prior same-axis sub-threshold strikes. In every trimmed cell firing jumps sharply from vc=1 to vc=2; untrimmed cells stay low at both — the threshold-laxity signature.

Cell	variant	vc=1	vc=2
A1	untrimmed	0.063	0.352
A1	trimmed	0.654	0.912
A5	untrimmed	0.031	0.207
A5	trimmed	0.384	0.736

Cell	variant	vc=1	vc=2
A6	untrimmed	0.050	0.264
A6	trimmed	0.497	0.786
A7	untrimmed	0.069	0.358
A7	trimmed	0.497	0.887

Table B.2 — Attribution under distraction (§5.4). `fire_correct` records (primary axis X at the third strike, distractor axis Y sub-threshold), typed cells only (n=141 each). The gate names the threshold axis X on 0.96–0.98 of fires and is pulled to the distractor under 2%.

Cell	marker	fire_rate (X at 3)	correct-axis (X)	wrong-axis (Y)	no-axis fires
A1	typed	0.837	0.975	0.009	0
A5	typed	0.766	0.963	0.018	0

Code and Data Availability

Code, training/evaluation scripts, configuration, and the synthetic datasets are available at <https://github.com/cch-ai922/tutor-train>. Model weights and large training outputs are not included; the base model is Qwen3.5-0.8B-Base. The released datasets are model-generated (teacher-distilled) and contain no personal data.

References

- Bhattachamishra, Satwik, Kabir Ahuja, and Navin Goyal. 2020. “On the Ability and Limitations of Transformers to Recognize Formal Languages.” *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 7096–116.
- Dettmers, Tim, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. 2023. “QLoRA: Efficient Fine-tuning of Quantized LLMs.” *Advances in Neural Information Processing Systems (NeurIPS)*.
- Dong, Yixin, Charlie F. Ruan, Yaxing Cai, et al. 2024. “XGrammar: Flexible and Efficient Structured Generation Engine for Large Language Models.” *arXiv Preprint arXiv:2411.15100*.
- El-Yaniv, Ran, and Yair Wiener. 2010. “On the Foundations of Noise-Free Selective Classification.” *Journal of Machine Learning Research* 11.
- Geifman, Yonatan, and Ran El-Yaniv. 2017. “Selective Classification for Deep Neural Networks.” *Advances in Neural Information Processing Systems (NeurIPS)*.
- Geirhos, Robert, Jörn-Henrik Jacobsen, Claudio Michaelis, et al. 2020. “Shortcut Learning in Deep Neural Networks.” *Nature Machine Intelligence* 2 (11): 665–73.
- Gururangan, Suchin, Swabha Swayamdipta, Omer Levy, Roy Schwartz, Samuel R. Bowman, and Noah A. Smith. 2018. “Annotation Artifacts in Natural Language Inference Data.” *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 107–12.

- Hu, Edward J., Yelong Shen, Phillip Wallis, et al. 2022. "LoRA: Low-Rank Adaptation of Large Language Models." *International Conference on Learning Representations (ICLR)*.
- Lightman, Hunter, Vineet Kosaraju, Yura Burda, et al. 2024. "Let's Verify Step by Step." *International Conference on Learning Representations (ICLR)*.
- Liu, Weiwen, Xu Huang, Xingshan Zeng, et al. 2025. "ToolACE: Winning the Points of LLM Function Calling." *International Conference on Learning Representations (ICLR)*.
- Liu, Zuxin, Thai Hoang, Jianguo Zhang, et al. 2024. "APIGen: Automated Pipeline for Generating Verifiable and Diverse Function-Calling Datasets." *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*.
- McCoy, Tom, Ellie Pavlick, and Tal Linzen. 2019. "Right for the Wrong Reasons: Diagnosing Syntactic Heuristics in Natural Language Inference." *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, 3428–48.
- Muennighoff, Niklas, Alexander M. Rush, Boaz Barak, et al. 2023. "Scaling Data-Constrained Language Models." *Advances in Neural Information Processing Systems (NeurIPS)*.
- Newman, Benjamin, John Hewitt, Percy Liang, and Christopher D. Manning. 2020. "The EOS Decision and Length Extrapolation." *Proceedings of the Third BlackboxNLP Workshop on Analyzing and Interpreting Neural Networks for NLP*.
- Nye, Maxwell, Anders Johan Andreassen, Guy Gur-Ari, et al. 2021. "Show Your Work: Scratchpads for Intermediate Computation with Language Models." *arXiv Preprint arXiv:2112.00114*.
- Park, Kanghee, Jiayu Wang, Taylor Berg-Kirkpatrick, Nadia Polikarpova, and Loris D'Antoni. 2024. "Grammar-Aligned Decoding." *Advances in Neural Information Processing Systems (NeurIPS)*.
- Qin, Yujia, Shihao Liang, Yining Ye, et al. 2024. "ToolLLM: Facilitating Large Language Models to Master 16000+ Real-World APIs." *International Conference on Learning Representations (ICLR)*.
- Rafailov, Rafael, Archit Sharma, Eric Mitchell, Christopher D. Manning, Stefano Ermon, and Chelsea Finn. 2023. "Direct Preference Optimization: Your Language Model Is Secretly a Reward Model." *Advances in Neural Information Processing Systems (NeurIPS)*.
- Raffel, Colin, Noam Shazeer, Adam Roberts, et al. 2020. "Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer." *Journal of Machine Learning Research* 21.
- Schick, Timo, Jane Dwivedi-Yu, Roberto Dessì, et al. 2023. "Toolformer: Language Models Can Teach Themselves to Use Tools." *Advances in Neural Information Processing Systems (NeurIPS)*.
- Stern, Mitchell, William Chan, Jamie Kiros, and Jakob Uszkoreit. 2019. "Insertion Transformer: Flexible Sequence Generation via Insertion Operations." *International Conference on Machine Learning (ICML)*.
- Wei, Jason, Xuezhi Wang, Dale Schuurmans, et al. 2022. "Chain-of-Thought Prompting Elicits Reason-

- ing in Large Language Models.” *Advances in Neural Information Processing Systems (NeurIPS)*.
- Willard, Brandon T., and Rémi Louf. 2023. “Efficient Guided Generation for Large Language Models.” *arXiv Preprint arXiv:2307.09702*.
- Wolf, Thomas, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, et al. 2020. “Transformers: State-of-the-Art Natural Language Processing.” *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*.
- Yao, Shunyu, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. 2024. “ τ -bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains.” *arXiv Preprint arXiv:2406.12045*.
- Zheng, Chujie, Zhenru Zhang, Beichen Zhang, et al. 2024. “ProcessBench: Identifying Process Errors in Mathematical Reasoning.” *arXiv Preprint arXiv:2412.06559*.
- Zhou, Chunting, Pengfei Liu, Puxin Xu, et al. 2023. “LIMA: Less Is More for Alignment.” *Advances in Neural Information Processing Systems (NeurIPS)*.